

AC - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4157 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4118 values"
## [1] "39 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	521	29	126	NA	NA	676
2	60	NA	413	92	104	669
3	297	NA	160	202	15	674
4	290	NA	224	62	37	613
5	222	NA	202	64	36	524
6	195	1	130	67	22	415
7	165	NA	96	28	18	307
8	84	NA	49	25	3	161
9	70	NA	2	NA	7	79

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7707101	0.0428994	0.1863905	NA	NA	676
2	0.0896861	NA	0.6173393	0.1375187	0.1554559	669
3	0.4406528	NA	0.2373887	0.2997033	0.0222552	674
4	0.4730832	NA	0.3654160	0.1011419	0.0603589	613
5	0.4236641	NA	0.3854962	0.1221374	0.0687023	524
6	0.4698795	0.0024096	0.3132530	0.1614458	0.0530120	415
7	0.5374593	NA	0.3127036	0.0912052	0.0586319	307
8	0.5217391	NA	0.3043478	0.1552795	0.0186335	161
9	0.8860759	NA	0.0253165	NA	0.0886076	79

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position
```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
      color=syll_component,
      linetype = syll_component,
      shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=s
```

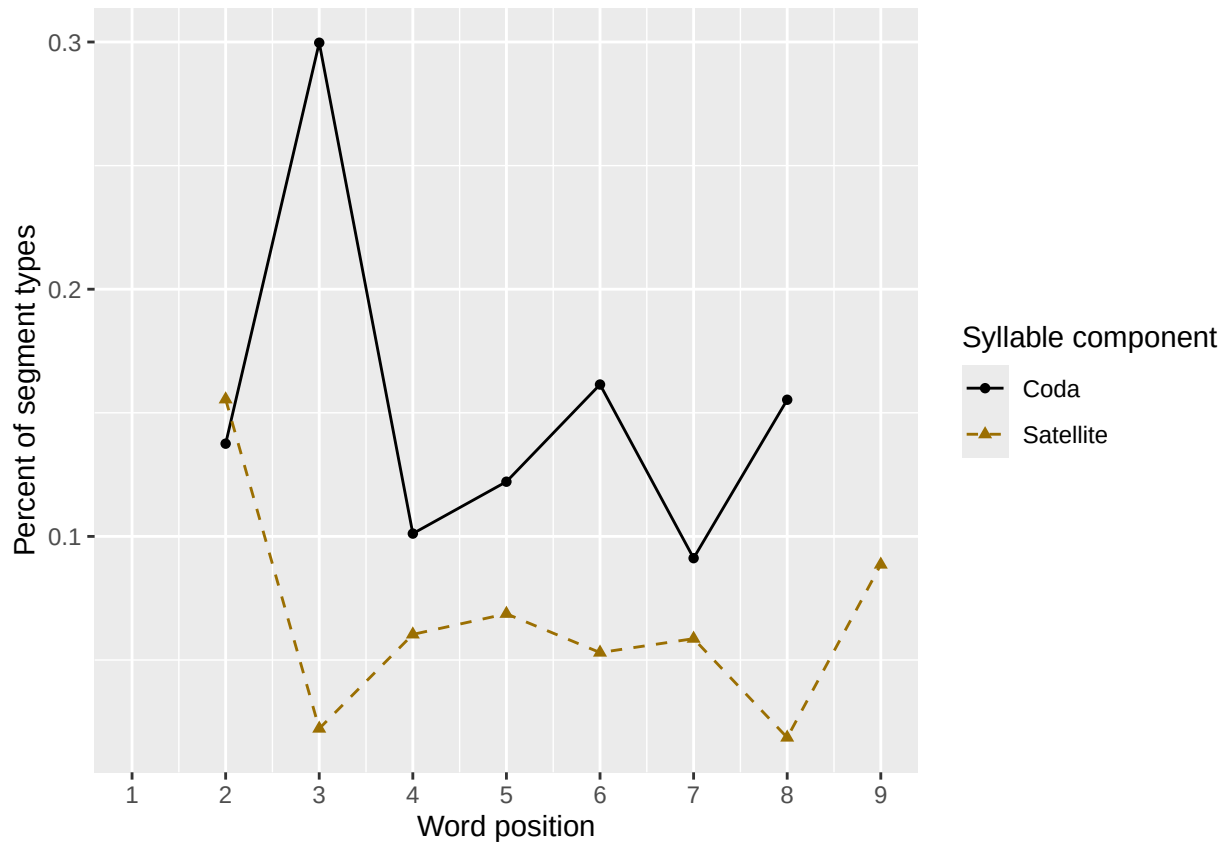
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen  `1`  `2`  `3`  `4`  `5`  `6`  `7`  `8`  `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 1     1     1    NA    NA    NA    NA    NA    NA
## 2      5 0.912 0.967 0.978 0.989 NA    NA    NA    NA    NA
## 3      6 0.947 0.965 0.956 0.956 0.956 NA    NA    NA    NA
## 4      7 0.935 0.972 0.972 0.981 0.991 0.963 NA    NA    NA
## 5      8 0.932 0.979 0.972 0.943 0.951 0.945 0.945 NA    NA
## 6      9 0.878 0.962 0.963 0.939 0.975 0.963 0.976 0.988 NA
## 7     10 0.936 0.961 0.912 0.949 0.911 0.888 0.888 0.937 0.949

```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))

```

```
pos_len_N_table
```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen  `1`  `2`  `3`  `4`  `5`  `6`  `7`  `8`  `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4   58   58   58   NA   NA   NA   NA   NA   NA
## 2      5   91   91   91   91   NA   NA   NA   NA   NA
## 3      6  113  113  113  113  113   NA   NA   NA   NA
## 4      7  108  107  107  108  107  108   NA   NA   NA
## 5      8  146  143  143  141  144  146  145   NA   NA
## 6      9   82   80   82   82   81   81   82   82   NA
## 7     10   78   77   80   78   79   80   80   79   79

```

```

obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")

```

```
pred_linetypes <- "dashed"
```

```
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
```

```
pos_len_summary$pos <- factor(pos_len_summary$pos)
```

```
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
```

```

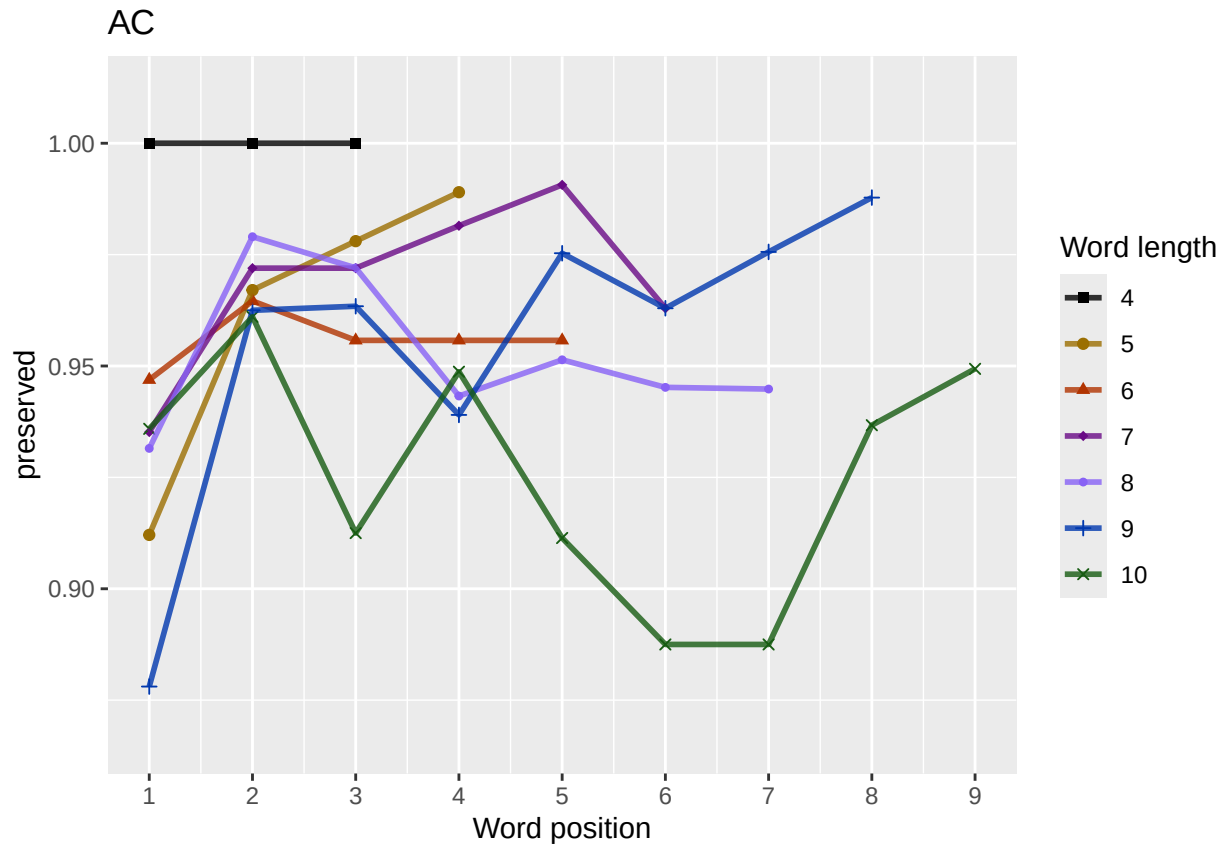
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
                      "preserved ~ stimlen",
                      "preserved ~ pos",
                      "preserved ~ stimlen + pos",
                      "preserved ~ stimlen*pos",
                      "preserved ~ I(pos^2)+pos",
                      "preserved ~ stimlen + I(pos^2) + pos",
                      "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 5
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      3.44405      -0.08842      0.40782      -0.04034
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1512  AIC: 1520
## log likelihood:  -756.1983
## Nagelkerke R2:  0.016786
## % pres/err predicted correctly:  -357.7338
## % of predictable range [ (model-null)/(1-null) ]:  0.005316639
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.49663      -0.21192      0.05369
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4115 Residual
## Null Deviance:      1534
## Residual Deviance: 1515  AIC: 1521
## log likelihood:  -757.5386
## Nagelkerke R2:  0.01470298
## % pres/err predicted correctly:  -358.0155
## % of predictable range [ (model-null)/(1-null) ]:  0.0045356
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.5008      -0.1862
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1517  AIC: 1521
## log likelihood:  -758.656
## Nagelkerke R2:  0.01296547
## % pres/err predicted correctly:  -358.1971
## % of predictable range [ (model-null)/(1-null) ]:  0.004032257
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos

```

```

##      4.17618      -0.20241      -0.01833      0.21139
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1514 AIC: 1522
## log likelihood:  -756.782
## Nagelkerke R2:  0.015879
## % pres/err predicted correctly:  -357.8912
## % of predictable range [ (model-null)/(1-null) ]:  0.004880213
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      1.92349      0.07450      -0.17953      1.63133      0.01892      -0.01892
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4112 Residual
## Null Deviance:      1534
## Residual Deviance: 1510 AIC: 1522
## log likelihood:  -754.8334
## Nagelkerke R2:  0.01890564
## % pres/err predicted correctly:  -357.5722
## % of predictable range [ (model-null)/(1-null) ]:  0.005764726
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)
##      3.034
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4117 Residual
## Null Deviance:      1534
## Residual Deviance: 1534 AIC: 1536
## log likelihood:  -766.9749
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -359.6513
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)      pos
##      2.66204      -0.02842      0.24231
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4115 Residual

```



```
## Null Deviance:      1534
## Residual Deviance: 1530 AIC: 1536
## log likelihood:    -765.1121
## Nagelkerke R2:     0.00290786
## % pres/err predicted correctly: -359.3219
## % of predictable range [ (model-null)/(1-null) ]:  0.0009133414
## *****
## model index:      3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.054468      -0.005235
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1534 AIC: 1538
## log likelihood:    -766.9636
## Nagelkerke R2:     1.76753e-05
## % pres/err predicted correctly: -359.6492
## % of predictable range [ (model-null)/(1-null) ]:  5.889667e-06
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	1520.397	0.000000	0.000000	0.0291308	0.0016786	0.444050	-	0.4078233	-	NA	NA
preserved ~ stimlen + pos	1521.077	0.680762	0.711490	0.0207265	0.0147034	0.496626	-	0.0536921	NA	NA	NA
preserved ~ stimlen	1521.312	0.915523	0.386326	0.0118431	0.0012963	0.500787	-	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2)	1521.561	1.167484	0.255780	0.0116249	0.0015879	0.176178	-	0.2113924	NA	-	NA
+ pos							0.2024092			0.0183265	

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1521.667	7.270332	33529847	41543488	0.189036	234890.074500	0.6313284	-	-	0.0189164	0.1701180.1795293
preserved ~ 1	1535.951	15.553380	0.000419	0.000122	0.000000	0.0034393	NA	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	1536.224	15.827730	0.000366	0.000106	0.000290	29662040	NA	0.2423116	NA	-	NA
preserved ~ pos	1537.927	17.530741	0.000156	0.000045	0.000013	7054468	NA	-	NA	0.0284212	NA
								0.0052352			

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen * pos"
```

```
print(BestLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      3.44405      -0.08842       0.40782      -0.04034
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1512  AIC: 1520
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4 0.966 0.973 0.979 NA     NA     NA     NA     NA     NA
## 2     5 0.961 0.968 0.974 0.979 NA     NA     NA     NA     NA
## 3     6 0.956 0.962 0.968 0.973 0.977 NA     NA     NA     NA
## 4     7 0.950 0.956 0.961 0.965 0.969 0.973 NA     NA     NA
## 5     8 0.944 0.948 0.952 0.956 0.959 0.963 0.966 NA     NA
## 6     9 0.937 0.939 0.942 0.944 0.946 0.949 0.951 0.953 NA
## 7    10 0.929 0.929 0.929 0.929 0.930 0.930 0.930 0.931 0.931
```

```

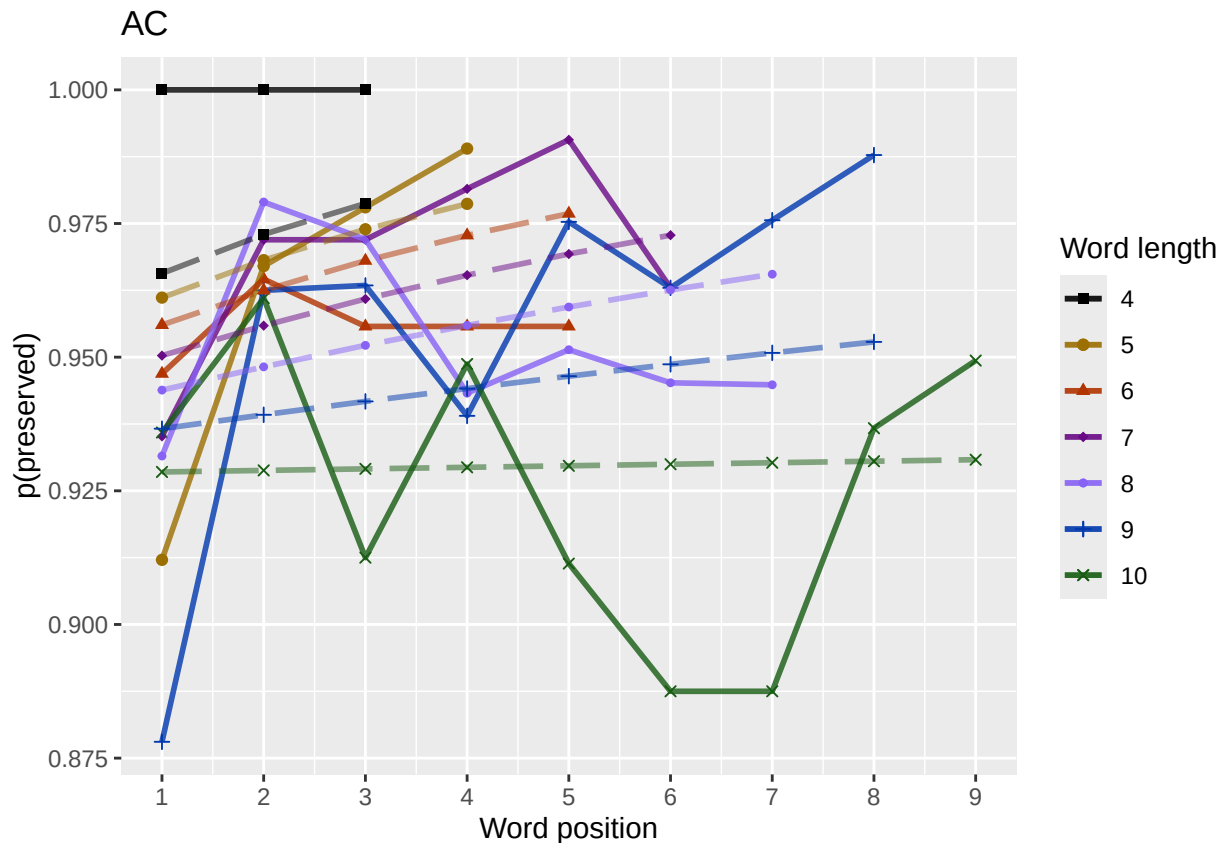
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

```

```

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      6  679

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 6 / 679 = 0.88 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
##   (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stiml
##   0.96117      0.17738      -0.31715      2.60102      0.03436      -0
##
## Degrees of Freedom: 4100 Total (i.e. Null); 4095 Residual
## Null Deviance: 1428
## Residual Deviance: 1397 AIC: 1409
## log likelihood: -698.283
## Nagelkerke R2: 0.02570578
## % pres/err predicted correctly: -326.2216
## % of predictable range [ (model-null)/(1-null) ]: 0.007132569

```

```

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.5627      -0.2299      0.1002
##
## Degrees of Freedom: 4100 Total (i.e. Null); 4098 Residual
## Null Deviance:      1428
## Residual Deviance: 1406 AIC: 1412
## log likelihood: -702.8665
## Nagelkerke R2: 0.01815162
## % pres/err predicted correctly: -326.822
## % of predictable range [ (model-null)/(1-null) ]: 0.005310907
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##      3.62959      -0.12049      0.42602      -0.03718
##
## Degrees of Freedom: 4100 Total (i.e. Null); 4097 Residual
## Null Deviance:      1428
## Residual Deviance: 1404 AIC: 1412
## log likelihood: -701.8815
## Nagelkerke R2: 0.01977634
## % pres/err predicted correctly: -326.6683
## % of predictable range [ (model-null)/(1-null) ]: 0.00577717
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      4.30586      -0.22263      -0.01564      0.23196
##
## Degrees of Freedom: 4100 Total (i.e. Null); 4097 Residual
## Null Deviance:      1428
## Residual Deviance: 1405 AIC: 1413
## log likelihood: -702.3909
## Nagelkerke R2: 0.01893624
## % pres/err predicted correctly: -326.7469
## % of predictable range [ (model-null)/(1-null) ]: 0.005538814
## *****
## model index: 2
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.5816      -0.1846
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4099 Residual
## Null Deviance:      1428
## Residual Deviance: 1413 AIC: 1417
## log likelihood: -706.3825
## Nagelkerke R2:  0.01234521
## % pres/err predicted correctly: -327.4078
## % of predictable range [ (model-null)/(1-null) ]:  0.003533271
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      3.129
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4100 Residual
## Null Deviance:      1428
## Residual Deviance: 1428 AIC: 1430
## log likelihood: -713.8381
## Nagelkerke R2:  0
## % pres/err predicted correctly: -328.5723
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.63286      -0.02666      0.26628
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4098 Residual
## Null Deviance:      1428
## Residual Deviance: 1424 AIC: 1430
## log likelihood: -711.8623
## Nagelkerke R2:  0.003275983
## % pres/err predicted correctly: -328.2214
## % of predictable range [ (model-null)/(1-null) ]:  0.001064687
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

```
## Coefficients:
## (Intercept)          pos
##      2.98057      0.03966
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4099 Residual
## Null Deviance:      1428
## Residual Deviance: 1427  AIC: 1431
## log likelihood:  -713.2626
## Nagelkerke R2:  0.000954606
## % pres/err predicted correctly:  -328.4747
## % of predictable range [ (model-null)/(1-null) ]:  0.0002962498
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1408.566	0.000000	0.000000	0.0646989	0.0257058	99611748	148.1773772	36010229	-	-	0.0343611
									0.2758270	0.3171481	
preserved ~ stimlen + pos	1411.733	3.166859	0.2052699	0.1328075	0.0181516	5627275	-	0.1002250	NA	NA	NA
							0.2299305				
preserved ~ stimlen * pos	1411.763	3.197005	0.2021999	0.1130820	0.0197763	6295913	-	0.4260247	-	NA	NA
							0.1204890	0.0371775			
preserved ~ stimlen + I(pos^2) + pos	1412.782	4.215679	0.1215002	0.0278609	0.0189362	3058566	-	0.2319575	NA	-	NA
							0.2226264			0.0156355	
preserved ~ stimlen	1416.765	8.198940	0.0165805	0.0107280	0.0123442	5816381	-	NA	NA	NA	NA
							0.1845931				
preserved ~ 1	1429.675	21.110166	0.000026	0.000001	0.000000	001286457	NA	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	1429.723	21.158536	0.000025	0.000001	0.0003270	60328571	NA	0.2662764	NA	-	NA
										0.0266620	
preserved ~ pos	1430.523	21.959073	0.000017	0.000001	0.0000954	9805744	NA	0.0396616	NA	NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

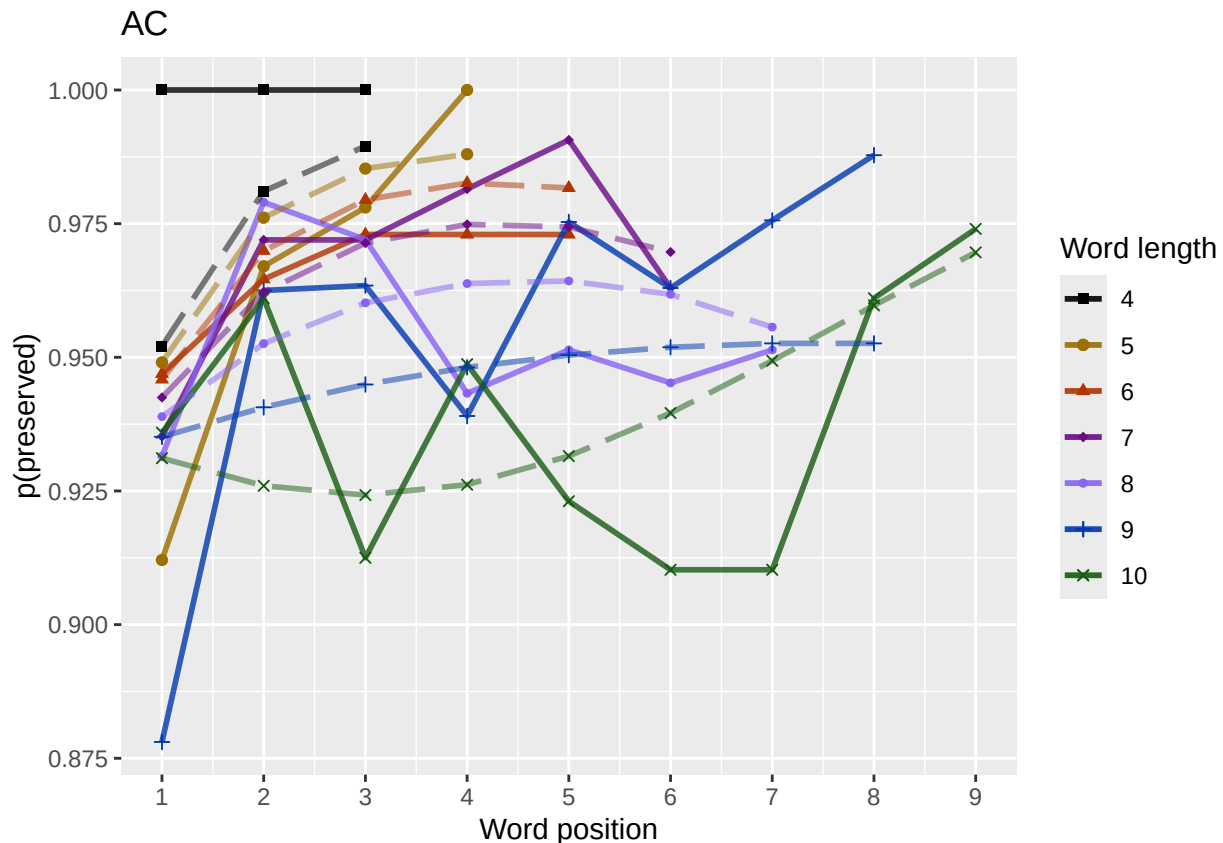
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.952 0.981 0.990 NA      NA      NA      NA      NA      NA
## 2      5 0.949 0.976 0.985 0.988 NA      NA      NA      NA      NA
## 3      6 0.946 0.970 0.979 0.983 0.982 NA      NA      NA      NA
## 4      7 0.942 0.962 0.971 0.975 0.974 0.970 NA      NA      NA
## 5      8 0.939 0.953 0.960 0.964 0.964 0.962 0.956 NA      NA
## 6      9 0.935 0.941 0.945 0.948 0.950 0.952 0.953 0.953 NA
## 7     10 0.931 0.926 0.924 0.926 0.932 0.940 0.949 0.960 0.970
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen)) +
#   geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
# geom_point(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.87 - 1.01"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.008824499
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] 0.002585515
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

CurrentLabel<-"Average upward change after U minimum"
print(CurrentLabel)
print(OA_mean_pos_u_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

CurrentLabel<-"Proportion of average downward change"
prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
}
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stim
##      3.90344      -0.18490      0.59460      -0.02906      0.31454
##      log_freq:pos
##      0.14784
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4110 Residual
## Null Deviance:      1534
## Residual Deviance: 1480 AIC: 1496
## log likelihood: -740.1102
## Nagelkerke R2:  0.04168211
## % pres/err predicted correctly: -355.3409
## % of predictable range [ (model-null)/(1-null) ]:  0.01195165
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      pos      stimlen:log_freq
##      4.39970      -0.19525      0.85712      0.05371      -0.08413
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4113 Residual
## Null Deviance:      1534
## Residual Deviance: 1486 AIC: 1496
## log likelihood: -743.2144
## Nagelkerke R2:  0.03689363
## % pres/err predicted correctly: -355.7394
## % of predictable range [ (model-null)/(1-null) ]:  0.01084672
## *****
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stimlen:log
##      4.03811      -0.18398      0.85892      -0.02047      0.22942      -0
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4112 Residual
## Null Deviance:      1534
## Residual Deviance: 1485 AIC: 1497
## log likelihood: -742.2864
## Nagelkerke R2:  0.03832591
## % pres/err predicted correctly: -355.6048
## % of predictable range [ (model-null)/(1-null) ]:  0.01122005
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```

##      (Intercept)          stimlen      log_freq stimlen:log_freq
##      4.40367          -0.16951      0.85606      -0.08397
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1489 AIC: 1497
## log likelihood: -744.3264
## Nagelkerke R2:  0.03517638
## % pres/err predicted correctly: -355.9335
## % of predictable range [ (model-null)/(1-null) ]:  0.01030857
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen      log_freq          pos  stimlen:log_freq      log_fr
##      4.39717          -0.19988      0.85458      0.06453      -0.09125      0
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4112 Residual
## Null Deviance:      1534
## Residual Deviance: 1486 AIC: 1498
## log likelihood: -742.8888
## Nagelkerke R2:  0.03739619
## % pres/err predicted correctly: -355.7451
## % of predictable range [ (model-null)/(1-null) ]:  0.01083088
## *****
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen      I(pos^2)          pos      log_freq  I(pos
##      3.67047          -0.14751      -0.03060      0.31123      -0.04892
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4111 Residual
## Null Deviance:      1534
## Residual Deviance: 1489 AIC: 1503
## log likelihood: -744.7038
## Nagelkerke R2:  0.03459347
## % pres/err predicted correctly: -356.0022
## % of predictable range [ (model-null)/(1-null) ]:  0.01011804
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen          pos      log_freq
##      4.16725          -0.16270      0.05333      0.17215
##

```

```

## Degrees of Freedom: 4117 Total (i.e. Null); 4114 Residual
## Null Deviance: 1534
## Residual Deviance: 1497 AIC: 1505
## log likelihood: -748.4406
## Nagelkerke R2: 0.02881519
## % pres/err predicted correctly: -356.6593
## % of predictable range [ (model-null)/(1-null) ]: 0.008295964
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq
## 4.1715 -0.1372 0.1723
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4115 Residual
## Null Deviance: 1534
## Residual Deviance: 1499 AIC: 1505
## log likelihood: -749.5369
## Nagelkerke R2: 0.02711799
## % pres/err predicted correctly: -356.8305
## % of predictable range [ (model-null)/(1-null) ]: 0.007821294
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq
## 3.83015 -0.15241 -0.01919 0.21822 0.17280
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4113 Residual
## Null Deviance: 1534
## Residual Deviance: 1495 AIC: 1505
## log likelihood: -747.6201
## Nagelkerke R2: 0.03008485
## % pres/err predicted correctly: -356.572
## % of predictable range [ (model-null)/(1-null) ]: 0.008538065
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 4.170993 -0.161253 0.048931 0.192713 -0.005333
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4113 Residual
## Null Deviance: 1534
## Residual Deviance: 1497 AIC: 1507

```

```

## log likelihood: -748.3978
## Nagelkerke R2: 0.02888148
## % pres/err predicted correctly: -356.6428
## % of predictable range [ (model-null)/(1-null) ]: 0.008341864
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos log_freq I(pos^2):log_freq
## 2.571488 -0.038143 0.335353 -0.005205 -0.018776
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4112 Residual
## Null Deviance: 1534
## Residual Deviance: 1497 AIC: 1509
## log likelihood: -748.6836
## Nagelkerke R2: 0.02843905
## % pres/err predicted correctly: -356.4914
## % of predictable range [ (model-null)/(1-null) ]: 0.008761536
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq
## 3.05008 0.01325 0.20339
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4115 Residual
## Null Deviance: 1534
## Residual Deviance: 1507 AIC: 1513
## log likelihood: -753.5258
## Nagelkerke R2: 0.02093505
## % pres/err predicted correctly: -357.3169
## % of predictable range [ (model-null)/(1-null) ]: 0.006472592
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq pos:log_freq
## 3.080413 0.004323 0.246372 -0.011321
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4114 Residual
## Null Deviance: 1534
## Residual Deviance: 1507 AIC: 1515
## log likelihood: -753.3382
## Nagelkerke R2: 0.02122608
## % pres/err predicted correctly: -357.2603

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.006629607
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##    3.44405    -0.08842    0.40782    -0.04034
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4114 Residual
## Null Deviance: 1534
## Residual Deviance: 1512 AIC: 1520
## log likelihood: -756.1983
## Nagelkerke R2: 0.016786
## % pres/err predicted correctly: -357.7338
## % of predictable range [ (model-null)/(1-null) ]: 0.005316639
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    4.49663    -0.21192    0.05369
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4115 Residual
## Null Deviance: 1534
## Residual Deviance: 1515 AIC: 1521
## log likelihood: -757.5386
## Nagelkerke R2: 0.01470298
## % pres/err predicted correctly: -358.0155
## % of predictable range [ (model-null)/(1-null) ]: 0.0045356
## *****
## model index: 15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    4.5008    -0.1862
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4116 Residual
## Null Deviance: 1534
## Residual Deviance: 1517 AIC: 1521
## log likelihood: -758.656
## Nagelkerke R2: 0.01296547
## % pres/err predicted correctly: -358.1971
## % of predictable range [ (model-null)/(1-null) ]: 0.004032257
## *****
## model index: 20

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      4.17618      -0.20241      -0.01833      0.21139
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1514 AIC: 1522
## log likelihood: -756.782
## Nagelkerke R2:  0.015879
## % pres/err predicted correctly: -357.8912
## % of predictable range [ (model-null)/(1-null) ]:  0.004880213
## *****
## model index:  21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)          pos stimlen:I(pos^2)      stimlen:I(pos^2)
##      1.92349      0.07450      -0.17953      1.63133      0.01892      -0.01892
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4112 Residual
## Null Deviance:      1534
## Residual Deviance: 1510 AIC: 1522
## log likelihood: -754.8334
## Nagelkerke R2:  0.01890564
## % pres/err predicted correctly: -357.5722
## % of predictable range [ (model-null)/(1-null) ]:  0.005764726
## *****
## model index:  14
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      3.034
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4117 Residual
## Null Deviance:      1534
## Residual Deviance: 1534 AIC: 1536
## log likelihood: -766.9749
## Nagelkerke R2:  0
## % pres/err predicted correctly: -359.6513
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.66204      -0.02842      0.24231
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4115 Residual
## Null Deviance:      1534
## Residual Deviance: 1530 AIC: 1536
## log likelihood:  -765.1121
## Nagelkerke R2:  0.00290786
## % pres/err predicted correctly:  -359.3219
## % of predictable range [ (model-null)/(1-null) ]:  0.0009133414
## *****
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.054468      -0.005235
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1534 AIC: 1538
## log likelihood:  -766.9636
## Nagelkerke R2:  1.76753e-05
## % pres/err predicted correctly:  -359.6492
## % of predictable range [ (model-null)/(1-null) ]:  5.889667e-06
## *****

BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC Delta	AIC	AICw	NagR	Intercept	log_freq	stimlen	log_pos	log_freq	I(pos^2)	pos^2	log_freq	I(pos^2)	pos	I(pos^2)
preserved ~ stimlen *	1496.220000000000	1496.220000000000	1496.220000000000	0.1848971	0.0838780	0.5946045	0.3145352	0.1478437	NA	-	NA	NA	NA	NA	NA
log_freq + (I(pos^2) + pos) *															
log_freq															
preserved ~ stimlen *	1496.4208040110	1496.4208040110	1496.4208040110	0.1952497	0.0841296	0.8571241	0.0537043	NA	NA	NA	NA	NA	NA	NA	NA
log_freq + pos															
preserved ~ stimlen *	1496.5732082384	1496.5732082384	1496.5732082384	0.1839806	0.0842661	0.8589186	0.2294229	NA	-	NA	NA	NA	NA	NA	NA
log_freq + I(pos^2) + pos															
preserved ~ stimlen *	1496.6132078056	1496.6132078056	1496.6132078056	0.1695144	0.0839739	0.8560613	NA	NA	NA	NA	NA	NA	NA	NA	NA
log_freq															
preserved ~ stimlen *	1497.1758701459	1497.1758701459	1497.1758701459	0.1998831	0.0912486	0.8545818	0.0645281	0.0156838	NA	NA	NA	NA	NA	NA	NA
log_freq + pos *															
log_freq															
preserved ~ stimlen + (I(pos^2) + pos) *	1503.7087063276	1503.7087063276	1503.7087063276	0.1475148	0.09153	-	NA	0.3110342	0.3386	-	-	NA	NA	NA	NA
log_freq															
preserved ~ stimlen + pos + log_freq	1504.8860032730	1504.8860032730	1504.8860032730	0.1627021	0.172142	0.172142	0.0533348	NA	NA	NA	NA	NA	NA	NA	NA
log_freq															
preserved ~ stimlen + log_freq	1505.8753038710	1505.8753038710	1505.8753038710	0.1371922	0.1722017	0.1722017	NA	NA	NA	NA	NA	NA	NA	NA	NA
log_freq															
preserved ~ stimlen + I(pos^2) + pos + log_freq	1505.9409007100	1505.9409007100	1505.9409007100	0.1524105	0.1727078	0.1727078	0.2182240	NA	-	NA	NA	NA	NA	NA	NA
log_freq															
preserved ~ stimlen + pos *	1506.1706760305	1506.1706760305	1506.1706760305	0.1612526	0.1927126	0.1927126	0.0489307	NA	NA	NA	NA	NA	NA	NA	NA
log_freq															
preserved ~ (I(pos^2) + pos) *	1509.3674677309	1509.3674677309	1509.3674677309	0.0052054	-	NA	0.3350532	0.6643	-	-	NA	NA	NA	NA	NA
log_freq															
preserved ~ pos + log_freq	1513.0523010002	1513.0523010002	1513.0523010002	0.2033835	0.0132467	0.2033835	0.0132467	NA	NA	NA	NA	NA	NA	NA	NA

Model	AIC Delta	AIC	AICw	NagR ²	(Intercept)	stimlen	log_freq	log_pos	log_freq:I(pos)	I(pos^2)	pos	log_freq:I(pos)	stimlen:I(pos^2)
preserved ~ pos *	1514.1876	1514.1876	0.9800	0.2211	3.90344	NA	0.2463718	0.0043234	NA	NA	NA	NA	NA
log_freq								0.0113205					
preserved ~ stimlen * pos	1520.2977	1520.2977	0.0560	0.0163	3.90344	0.059	NA	NA	0.407823	NA	NA	NA	- NA
						0.0884160							0.0403441
preserved ~ stimlen + pos	1521.2785	1521.2785	0.0930	0.0001	3.90344	0.059	NA	NA	0.0536921	NA	NA	NA	NA
						0.2119165							
preserved ~ stimlen	1521.2512	1521.2512	0.0360	0.0129	3.90344	0.059	NA	NA	NA	NA	NA	NA	NA
						0.1862292							
preserved ~ stimlen + I(pos^2) + pos	1521.2544	1521.2544	0.0300	0.0158	3.90344	0.059	NA	NA	0.2113924	NA	- NA	NA	NA
						0.2024092					0.0183265		
preserved ~ stimlen * (I(pos^2) + pos)	1521.2574	1521.2574	0.0300	0.0179	3.90344	0.059	NA	1.6313284	NA	- NA	NA	- NA	0.0189164
										0.1795293			0.1701180
preserved ~ 1	1535.3572	1535.3572	0.0000	0.0000	3.90344	0.059	NA	NA	NA	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	1536.4240	1536.4240	0.0000	0.0296	3.90344	0.059	NA	NA	0.2423116	NA	- NA	NA	NA
											0.0284212		
preserved ~ pos	1537.9270	1537.9270	0.0000	0.0000	3.90344	0.059	NA	NA	- NA	NA	NA	NA	NA
									0.0052352				

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq"
```

```
print(BestFLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)
```

```
## 3.90344
```

```
## log_freq:pos
```

```
## 0.14784
```

```
##
```

```
## Degrees of Freedom: 4117 Total (i.e. Null); 4110 Residual
```

```
## Null Deviance: 1534
```

```
## Residual Deviance: 1480 AIC: 1496
```

```
# do a median split on frequency to plot hf/lf effects (analysis is continuous)
```

```
median_freq <- median(PosDat$log_freq)
```

```
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
```

```
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
```

```

LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFData,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitted")

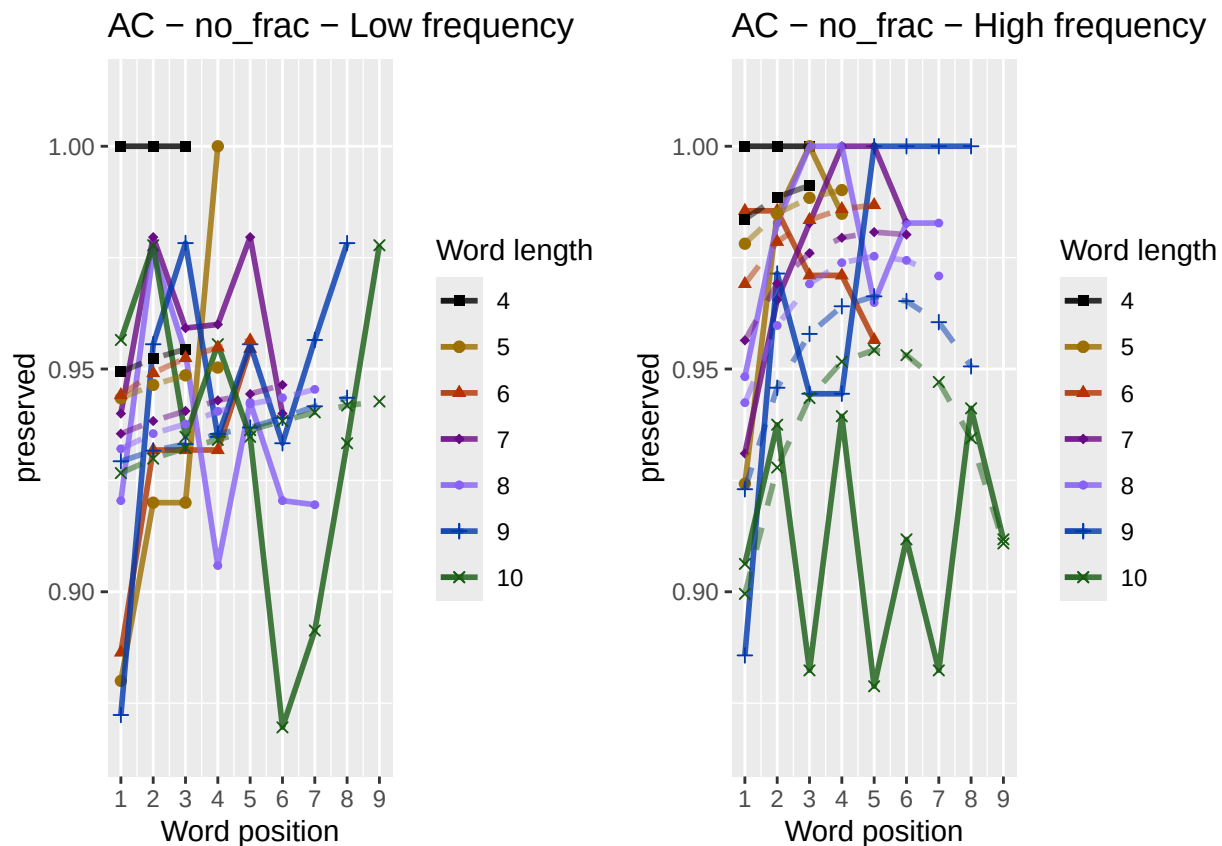
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

LF_Plot <- plot_len_pos_obs_predicted(LFData,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)

```



```

# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",

```

```

"preserved ~ pos",
"preserved ~ stimlen",
"preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.397      -1.138
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1380 AIC: 1384
## log likelihood: -690.1023
## Nagelkerke R2: 0.1178387
## % pres/err predicted correctly: -330.4231
## % of predictable range [ (model-null)/(1-null) ]: 0.08104281
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.5288      0.2253
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1501 AIC: 1505
## log likelihood: -750.5888
## Nagelkerke R2: 0.02548865
## % pres/err predicted correctly: -356.7423
## % of predictable range [ (model-null)/(1-null) ]: 0.008065893
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.5008      -0.1862
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1517 AIC: 1521
## log likelihood: -758.656

```

```

## Nagelkerke R2: 0.01296547
## % pres/err predicted correctly: -358.1971
## % of predictable range [ (model-null)/(1-null) ]: 0.004032257
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 3.034
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4117 Residual
## Null Deviance: 1534
## Residual Deviance: 1534 AIC: 1536
## log likelihood: -766.9749
## Nagelkerke R2: 0
## % pres/err predicted correctly: -359.6513
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.66204 -0.02842 0.24231
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4115 Residual
## Null Deviance: 1534
## Residual Deviance: 1530 AIC: 1536
## log likelihood: -765.1121
## Nagelkerke R2: 0.00290786
## % pres/err predicted correctly: -359.3219
## % of predictable range [ (model-null)/(1-null) ]: 0.0009133414
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.054468 -0.005235
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4116 Residual
## Null Deviance: 1534
## Residual Deviance: 1534 AIC: 1538
## log likelihood: -766.9636
## Nagelkerke R2: 1.76753e-05
## % pres/err predicted correctly: -359.6492
## % of predictable range [ (model-null)/(1-null) ]: 5.889667e-06

```



```
## *****
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1384.205	0.0000	1	1	0.1178387	3.396508	NA	-	NA	NA	NA
preserved ~ CumPres	1505.178	20.9731	0	0	0.0254887	2.528839	0.2252958	NA	NA	NA	NA
preserved ~ stimlen	1521.312	37.1074	0	0	0.0129654	1.500787	NA	NA	NA	NA	-
preserved ~ 1	1535.950	51.7453	0	0	0.0000000	0.034393	NA	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1536.224	52.0197	0	0	0.0029079	2.662040	NA	NA	-	0.2423116	NA
preserved ~ pos	1537.927	53.7227	0	0	0.0000173	0.054468	NA	NA	NA	-	NA
										0.0052352	

```
if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
}
```

```

AICValues <- c(BestMEModel$aic,RndModelAIC)
BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
BestMEModelRndDF <- rbind(BestMEModelRndDF,
                           data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

write.csv(BestMEModelRndDF,
           paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                  "_best_main_effects_model_with_random_cum_term.csv"),
           row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                         RunLabel,"_",CurTask,
                                         "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9722222	540
O	0.9438025	1904
P	0.6000000	30
S	0.9462810	242
V	0.9700428	1402

```

# main effects models for data without satellite positions

keep_components = c("0","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.468      -1.292
##

```

```

## Degrees of Freedom: 3845 Total (i.e. Null); 3844 Residual
## Null Deviance: 1356
## Residual Deviance: 1207 AIC: 1211
## log likelihood: -603.7114
## Nagelkerke R2: 0.1273254
## % pres/err predicted correctly: -284.6068
## % of predictable range [ (model-null)/(1-null) ]: 0.09046328
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 4.7516 -0.2075
##
## Degrees of Freedom: 3845 Total (i.e. Null); 3844 Residual
## Null Deviance: 1356
## Residual Deviance: 1338 AIC: 1342
## log likelihood: -668.9751
## Nagelkerke R2: 0.01551683
## % pres/err predicted correctly: -311.5845
## % of predictable range [ (model-null)/(1-null) ]: 0.004550831
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 2.7334 0.1715
##
## Degrees of Freedom: 3845 Total (i.e. Null); 3844 Residual
## Null Deviance: 1356
## Residual Deviance: 1340 AIC: 1344
## log likelihood: -669.9102
## Nagelkerke R2: 0.01388709
## % pres/err predicted correctly: -311.727
## % of predictable range [ (model-null)/(1-null) ]: 0.004097182
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.32982 -0.05515
##
## Degrees of Freedom: 3845 Total (i.e. Null); 3844 Residual
## Null Deviance: 1356
## Residual Deviance: 1353 AIC: 1357

```

```
## log likelihood: -676.7345
## Nagelkerke R2: 0.001969343
## % pres/err predicted correctly: -312.8185
## % of predictable range [ (model-null)/(1-null) ]: 0.0006211359
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 3.111
##
## Degrees of Freedom: 3845 Total (i.e. Null); 3845 Residual
## Null Deviance: 1356
## Residual Deviance: 1356 AIC: 1358
## log likelihood: -677.8599
## Nagelkerke R2: -7.47443e-16
## % pres/err predicted correctly: -313.0135
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 3.0626 -0.0182 0.1068
##
## Degrees of Freedom: 3845 Total (i.e. Null); 3843 Residual
## Null Deviance: 1356
## Residual Deviance: 1352 AIC: 1358
## log likelihood: -676.0577
## Nagelkerke R2: 0.003153281
## % pres/err predicted correctly: -312.7181
## % of predictable range [ (model-null)/(1-null) ]: 0.0009407054
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1211.4230	0.0000	1	1	0.1273253	3.467570	NA	-	NA	NA	NA
preserved ~ stimlen	1341.9501	130.5274	0	0	0.0155168	1.751594	NA	NA	NA	NA	-
preserved ~ CumPres	1343.8201	132.3977	0	0	0.0138872	1.733405	0.1715374	NA	NA	NA	0.2075133

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ pos	1357.4694	46.0463	0	0	0.0019693	3.329824	NA	NA	NA	-	NA
										0.0551521	
preserved ~ 1	1357.7204	46.2970	0	0	0.0000000	0.111345	NA	NA	NA	NA	NA
preserved ~	1358.1154	46.6926	0	0	0.0031533	0.062641	NA	NA	-	0.1067847	NA
(I(pos^2) + pos)									0.0181953		

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.363      -1.259
##
## Degrees of Freedom: 3305 Total (i.e. Null); 3304 Residual
## Null Deviance:      1215
## Residual Deviance: 1102 AIC: 1106
## log likelihood: -550.7595
## Nagelkerke R2: 0.1095836
## % pres/err predicted correctly: -261.3215
## % of predictable range [ (model-null)/(1-null) ]: 0.07818065
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.6606      -0.2039
##
## Degrees of Freedom: 3305 Total (i.e. Null); 3304 Residual
## Null Deviance:      1215
## Residual Deviance: 1199 AIC: 1203
## log likelihood: -599.5262
## Nagelkerke R2: 0.01549868
```

```

## % pres/err predicted correctly: -282.2115
## % of predictable range [ (model-null)/(1-null) ]: 0.004771461
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.6867      0.2008
##
## Degrees of Freedom: 3305 Total (i.e. Null); 3304 Residual
## Null Deviance:      1215
## Residual Deviance: 1200 AIC: 1204
## log likelihood: -600.0983
## Nagelkerke R2: 0.01437844
## % pres/err predicted correctly: -282.3234
## % of predictable range [ (model-null)/(1-null) ]: 0.004378028
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      3.053
##
## Degrees of Freedom: 3305 Total (i.e. Null); 3305 Residual
## Null Deviance:      1215
## Residual Deviance: 1215 AIC: 1217
## log likelihood: -607.4234
## Nagelkerke R2: 3.610317e-16
## % pres/err predicted correctly: -283.5693
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.24208      -0.04803
##
## Degrees of Freedom: 3305 Total (i.e. Null); 3304 Residual
## Null Deviance:      1215
## Residual Deviance: 1213 AIC: 1217
## log likelihood: -606.6066
## Nagelkerke R2: 0.001606403
## % pres/err predicted correctly: -283.4219
## % of predictable range [ (model-null)/(1-null) ]: 0.0005179312
## *****

```

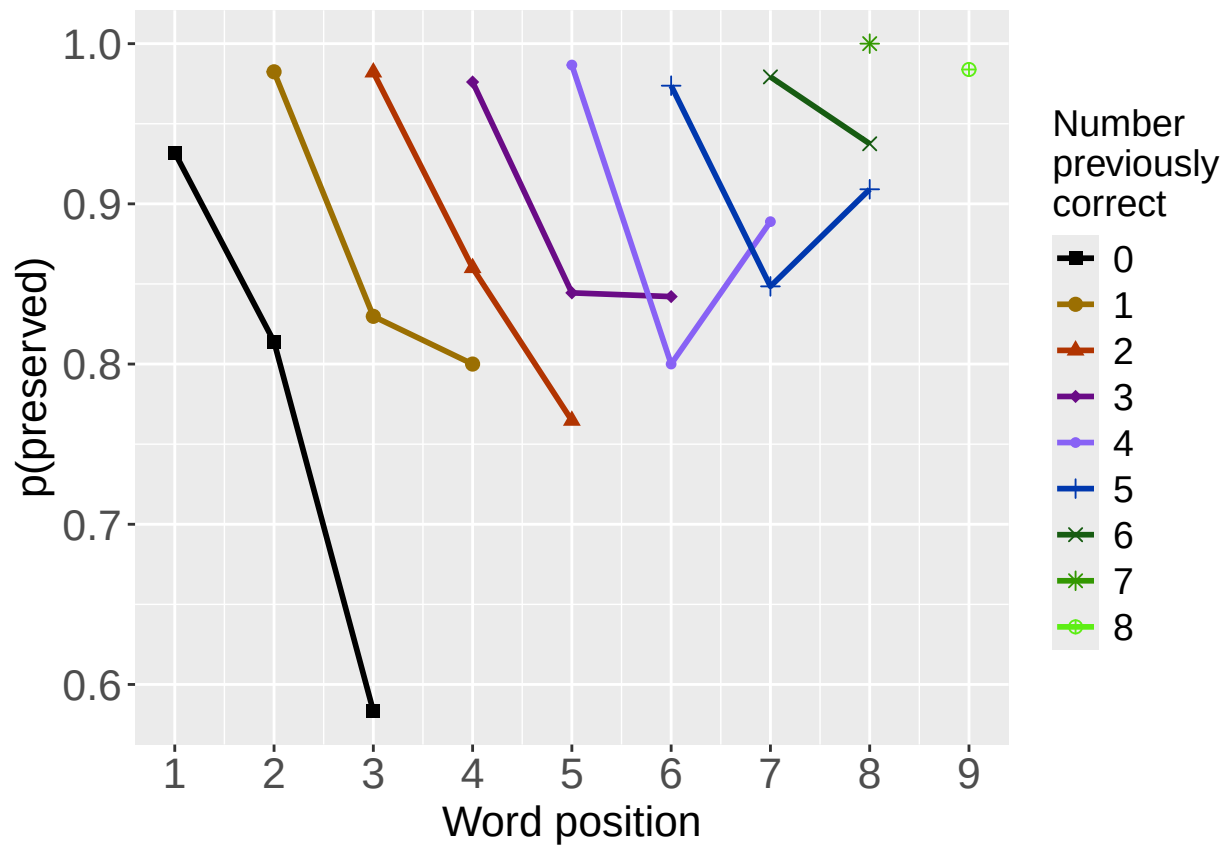
```
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.04620      -0.01394      0.07469
##
## Degrees of Freedom: 3305 Total (i.e. Null); 3303 Residual
## Null Deviance:      1215
## Residual Deviance: 1212 AIC: 1218
## log likelihood: -606.2279
## Nagelkerke R2: 0.002350875
## % pres/err predicted correctly: -283.3631
## % of predictable range [ (model-null)/(1-null) ]: 0.0007245464
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1105.519	0.00000	1	1	0.1095838	3.362977	NA	- 1.258582	NA	NA	NA
preserved ~ stimlen	1203.052	97.53335	0	0	0.0154987	4.660610	NA	NA	NA	NA	- 0.2038963
preserved ~ CumPres	1204.197	98.67750	0	0	0.0143782	4.686666	0.2008171	NA	NA	NA	NA
preserved ~ 1	1216.847	111.32762	0	0	0.0000000	0.053431	NA	NA	NA	NA	NA
preserved ~ pos	1217.213	111.69408	0	0	0.0016063	4.242078	NA	NA	NA	- 0.0480336	NA
preserved ~ (I(pos^2) + pos)	1218.456	112.93676	0	0	0.0023509	0.046197	NA	NA	- 0.0139351	0.0746930	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

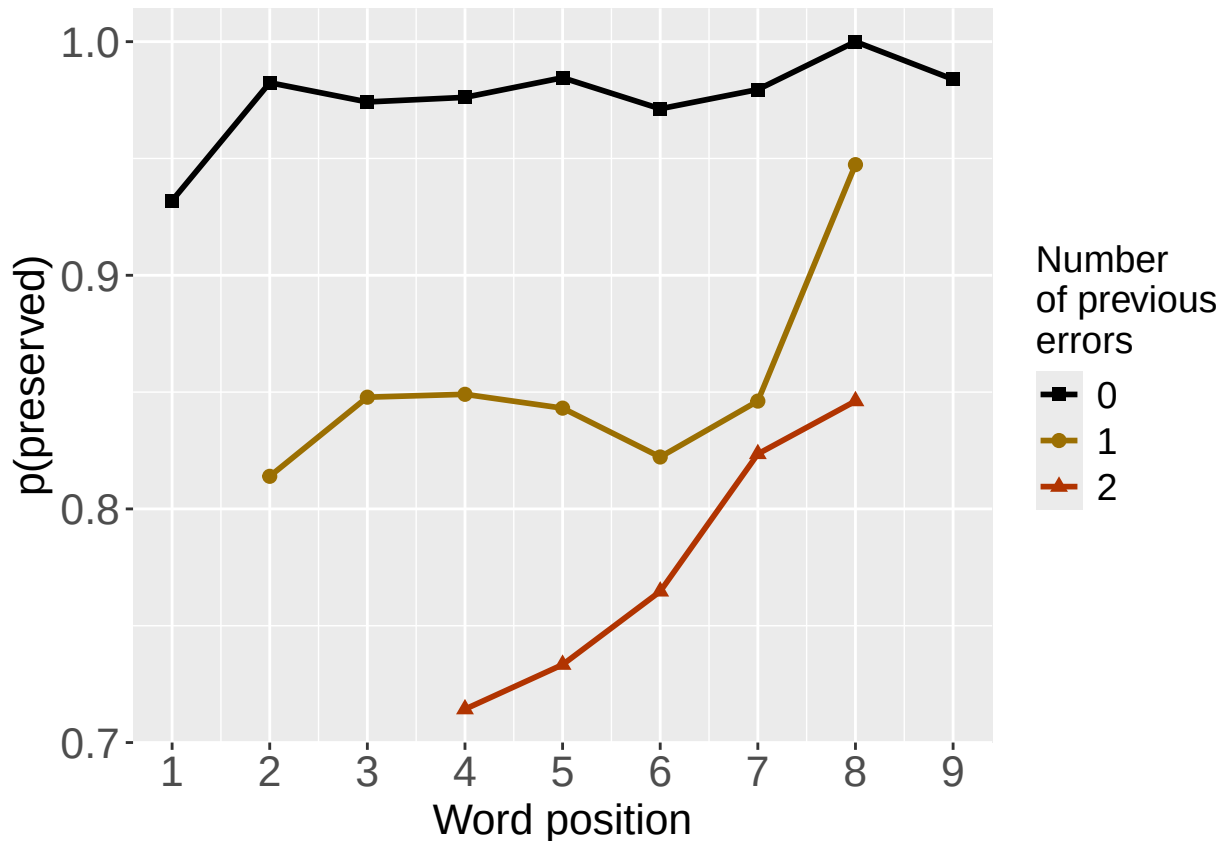
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

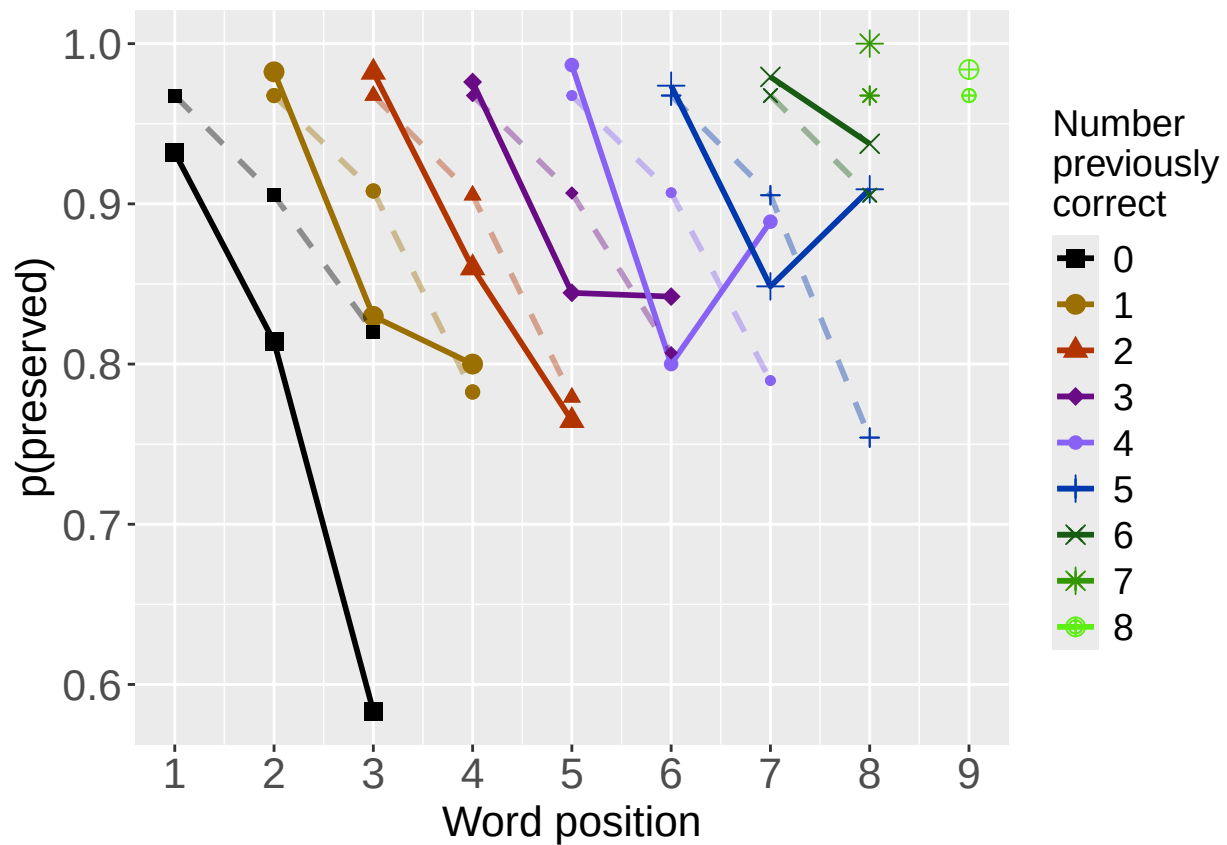
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

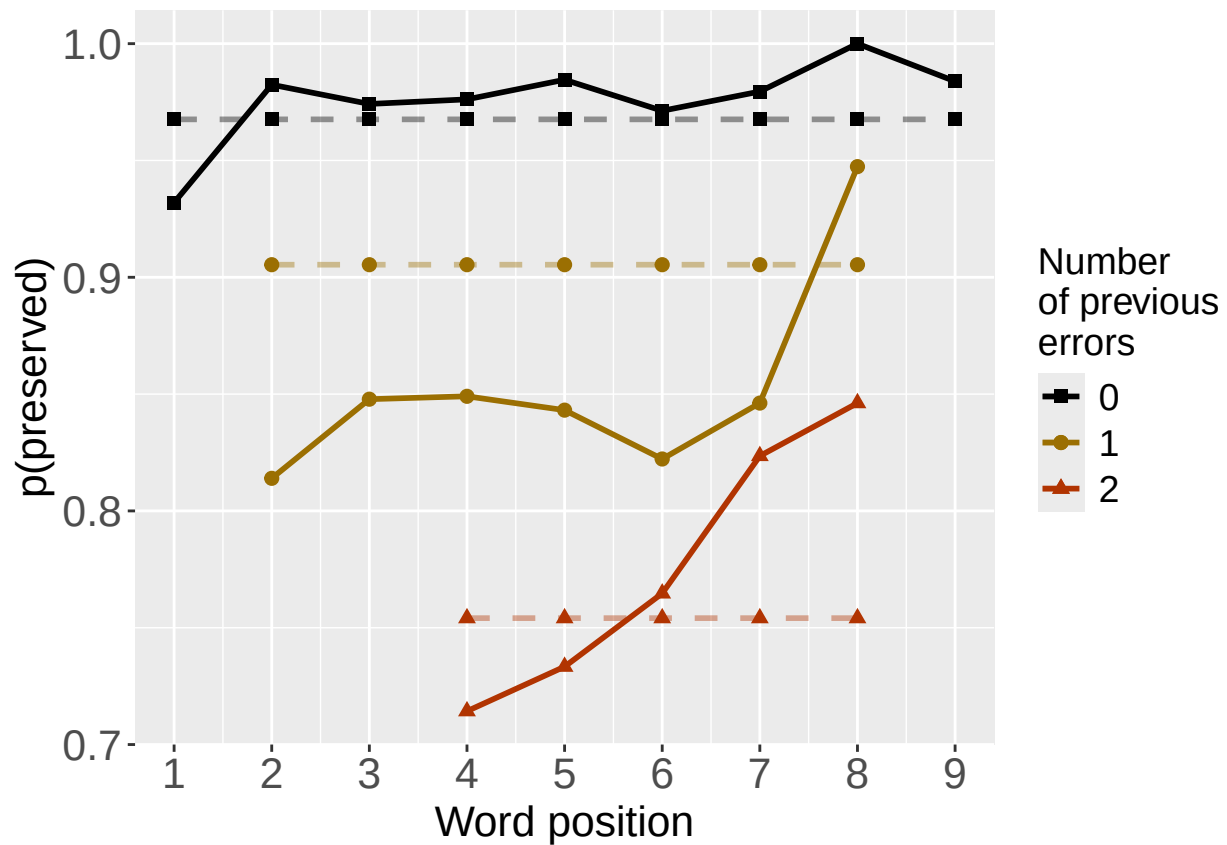
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    2.67386    -1.34708    -0.00761     0.25157
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1359  AIC: 1367
## log likelihood:  -679.6562
## Nagelkerke R2:  0.1335146
## % pres/err predicted correctly:  -328.3127
## % of predictable range [ (model-null)/(1-null) ]:  0.08689434
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.397      -1.138
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1380  AIC: 1384
## log likelihood:  -690.1023
## Nagelkerke R2:  0.1178387
## % pres/err predicted correctly:  -330.4231
## % of predictable range [ (model-null)/(1-null) ]:  0.08104281
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.66204      -0.02842      0.24231
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4115 Residual
## Null Deviance:      1534
## Residual Deviance: 1530  AIC: 1536
## log likelihood:  -765.1121
## Nagelkerke R2:  0.00290786
## % pres/err predicted correctly:  -359.3219
## % of predictable range [ (model-null)/(1-null) ]:  0.0009133414
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1367.312	0.00000	1.0000000	0.9997853	0.1335146	2.673862	-1.347080	-0.0076102	0.2515667
preserved ~ CumErr	1384.205	16.89209	0.0002147	0.0002147	0.1178387	3.396508	-1.137927	NA	NA
preserved ~ I(pos^2) + pos	1536.224	168.91174	0.0000000	0.0000000	0.0029079	2.662040	NA	-0.0284212	0.2423116

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 4.06508 -1.10222 -0.08692
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4115 Residual
## Null Deviance: 1534
## Residual Deviance: 1377 AIC: 1383
## log likelihood: -688.4878
## Nagelkerke R2: 0.1202667
## % pres/err predicted correctly: -330.1311
## % of predictable range [ (model-null)/(1-null) ]: 0.08185232
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 3.397 -1.138
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4116 Residual
## Null Deviance: 1534
## Residual Deviance: 1380 AIC: 1384
## log likelihood: -690.1023
## Nagelkerke R2: 0.1178387
## % pres/err predicted correctly: -330.4231
## % of predictable range [ (model-null)/(1-null) ]: 0.08104281
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      4.5008      -0.1862
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1517  AIC: 1521
## log likelihood:  -758.656
## Nagelkerke R2:  0.01296547
## % pres/err predicted correctly:  -358.1971
## % of predictable range [ (model-null)/(1-null) ]:  0.004032257
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	1382.976	0.000000	1.0000000	0.648974	0.1202667	4.065085	-	-
preserved ~ CumErr	1384.205	1.229064	0.5408939	0.351026	0.1178387	3.396508	-	NA
preserved ~ stimlen	1521.312	138.336513	0.0000000	0.000000	0.0129655	4.500787	NA	-
								0.1862292

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.8769      -1.1448      0.2324
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4115 Residual
## Null Deviance:      1534
## Residual Deviance: 1350  AIC: 1356
## log likelihood:  -675.0204
## Nagelkerke R2:  0.140446
## % pres/err predicted correctly:  -326.567
## % of predictable range [ (model-null)/(1-null) ]:  0.09173476
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.397      -1.138
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1380 AIC: 1384
## log likelihood: -690.1023
## Nagelkerke R2:  0.1178387
## % pres/err predicted correctly: -330.4231
## % of predictable range [ (model-null)/(1-null) ]:  0.08104281
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.5288      0.2253
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1501 AIC: 1505
## log likelihood: -750.5888
## Nagelkerke R2:  0.02548865
## % pres/err predicted correctly: -356.7423
## % of predictable range [ (model-null)/(1-null) ]:  0.008065893
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1356.041	0.00000	1e+00	0.9999992	0.1404460	2.876869	- 1.144771	0.2324205
preserved ~ CumErr	1384.205	28.16374	8e-07	0.0000008	0.1178387	3.396508	- 1.137927	NA
preserved ~ CumPres	1505.178	149.13682	0e+00	0.0000000	0.0254887	2.528839	NA	0.2252958

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      2.7694      -1.3509      0.1886
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4115 Residual
## Null Deviance:      1534
## Residual Deviance: 1360 AIC: 1366
## log likelihood: -679.7594
## Nagelkerke R2: 0.1333602
## % pres/err predicted correctly: -328.2503
## % of predictable range [ (model-null)/(1-null) ]: 0.08706732
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.397      -1.138
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1380 AIC: 1384
## log likelihood: -690.1023
## Nagelkerke R2: 0.1178387
## % pres/err predicted correctly: -330.4231
## % of predictable range [ (model-null)/(1-null) ]: 0.08104281
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.054468      -0.005235
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4116 Residual
## Null Deviance:      1534
## Residual Deviance: 1534 AIC: 1538
## log likelihood: -766.9636
## Nagelkerke R2: 1.76753e-05
## % pres/err predicted correctly: -359.6492
## % of predictable range [ (model-null)/(1-null) ]: 5.889667e-06
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	1365.519	0.00000	1.00e+00	0.9999124	0.1333602	2.769429	-	0.1886263
+ pos							1.350858	
preserved ~ CumErr	1384.205	18.68568	8.76e-05	0.0000876	0.1178387	3.396508	-	NA
							1.137927	
preserved ~ pos	1537.927	172.40835	0.00e+00	0.0000000	0.0000177	3.054468	NA	-
								0.0052352

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	1356.040	0.0000001	0.0000000	0.9999999	0.1404460	0.876869	-	NA	NA	NA	0.2324205
							1.144771				
preserved ~ CumErr + pos	1365.519	0.0000001	0.0000000	0.9999124	0.1333602	2.769429	-	NA	0.1886263	NA	NA
							1.350858				
preserved ~ CumErr + I(pos^2) + pos	1367.310	0.0000001	0.0000000	0.9997853	0.1335146	2.673862	-	-	0.2515667	NA	NA
							1.347080	0.0076102			
preserved ~ CumErr + stimlen	1382.970	0.0000001	0.0000000	0.6489740	0.1202667	0.65085	-	NA	NA	-	NA
							1.102223			0.0869183	
preserved ~ CumErr	1384.205	16.892080	0.0002147	0.0002147	0.1178387	3.396508	-	NA	NA	NA	NA
							1.137927				
preserved ~ CumErr	1384.205	1.2290640	0.5408939	0.3510260	0.1178387	3.396508	-	NA	NA	NA	NA
							1.137927				
preserved ~ CumErr	1384.205	8.163748	0.0000008	0.0000008	0.1178387	3.396508	-	NA	NA	NA	NA
							1.137927				
preserved ~ CumErr	1384.205	18.685680	0.0000876	0.0000876	0.1178387	3.396508	-	NA	NA	NA	NA
							1.137927				
preserved ~ CumPres	1505.178	149.1368220	0.0000000	0.0000000	0.0025488	2.528839	NA	NA	NA	NA	0.2252958
preserved ~ stimlen	1521.312	38.3365030	0.0000000	0.0000000	0.0012965	5.500787	NA	NA	NA	-	NA
										0.1862292	
preserved ~ I(pos^2) + pos	1536.224	168.91170	0.0000000	0.0000000	0.0002907	2.9662040	NA	-	0.2423116	NA	NA
							0.0284212				
preserved ~ pos	1537.927	172.40835	0.0000000	0.0000000	0.0000017	3.054468	NA	NA	-	NA	NA
										0.0052352	

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      3.9582      -1.0500      0.2777      -0.1475      0.1683
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4113 Residual
## Null Deviance:      1534
## Residual Deviance: 1321 AIC: 1331
## log likelihood: -660.2549
## Nagelkerke R2: 0.1624192
## % pres/err predicted correctly: -324.2595
## % of predictable range [ (model-null)/(1-null) ]: 0.09813288
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      2.8963      -1.0976      0.2469      0.1909
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1329 AIC: 1337
## log likelihood: -664.433
## Nagelkerke R2: 0.1562176
## % pres/err predicted correctly: -325.3066
## % of predictable range [ (model-null)/(1-null) ]: 0.09522951
## *****
## model index: 4

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      4.1850      -1.0726      0.2739      -0.1827
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1336 AIC: 1344
## log likelihood: -668.1719
## Nagelkerke R2: 0.1506572
## % pres/err predicted correctly: -324.8784
## % of predictable range [ (model-null)/(1-null) ]: 0.09641697
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.8769      -1.1448      0.2324
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4115 Residual
## Null Deviance:      1534
## Residual Deviance: 1350 AIC: 1356
## log likelihood: -675.0204
## Nagelkerke R2: 0.140446
## % pres/err predicted correctly: -326.567
## % of predictable range [ (model-null)/(1-null) ]: 0.09173476
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      3.034
##
## Degrees of Freedom: 4117 Total (i.e. Null);  4117 Residual
## Null Deviance:      1534
## Residual Deviance: 1534 AIC: 1536
## log likelihood: -766.9749
## Nagelkerke R2: 0
## % pres/err predicted correctly: -359.6513
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPre	log_freq	stimlen
preserved ~ CumErr + CumPres + stimlen + log_freq	1330.510	0.0000001	0.000000	0.959084	0.162419	2958183	-	0.277690	0.1682666	-
							1.049970			0.1474933
preserved ~ CumErr + CumPres + log_freq	1336.866	6.356099	0.041666	0.039962	0.156217	2896315	-	0.246943	0.1908965	NA
							1.097597			
preserved ~ CumErr + CumPres + stimlen	1344.344	3.833938	0.000990	0.000950	0.150657	2185016	-	0.273893	0.1826775	-
							1.072578			
preserved ~ CumErr + CumPres	1356.042	25.530937	0.000002	0.000000	0.1714044	20876869	-	0.232420	0.1826775	NA
							1.144771			
preserved ~ 1	1535.952	205.439986	0.000000	0.000000	0.000000	0.034393	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + stimlen + log_freq
##      Df Deviance   AIC
## CumErr    1   1446.9 1454.9
## CumPres    1   1362.0 1370.0
## log_freq    1   1336.3 1344.3
## stimlen    1   1328.9 1336.9
## <none>      1   1320.5 1330.5

```

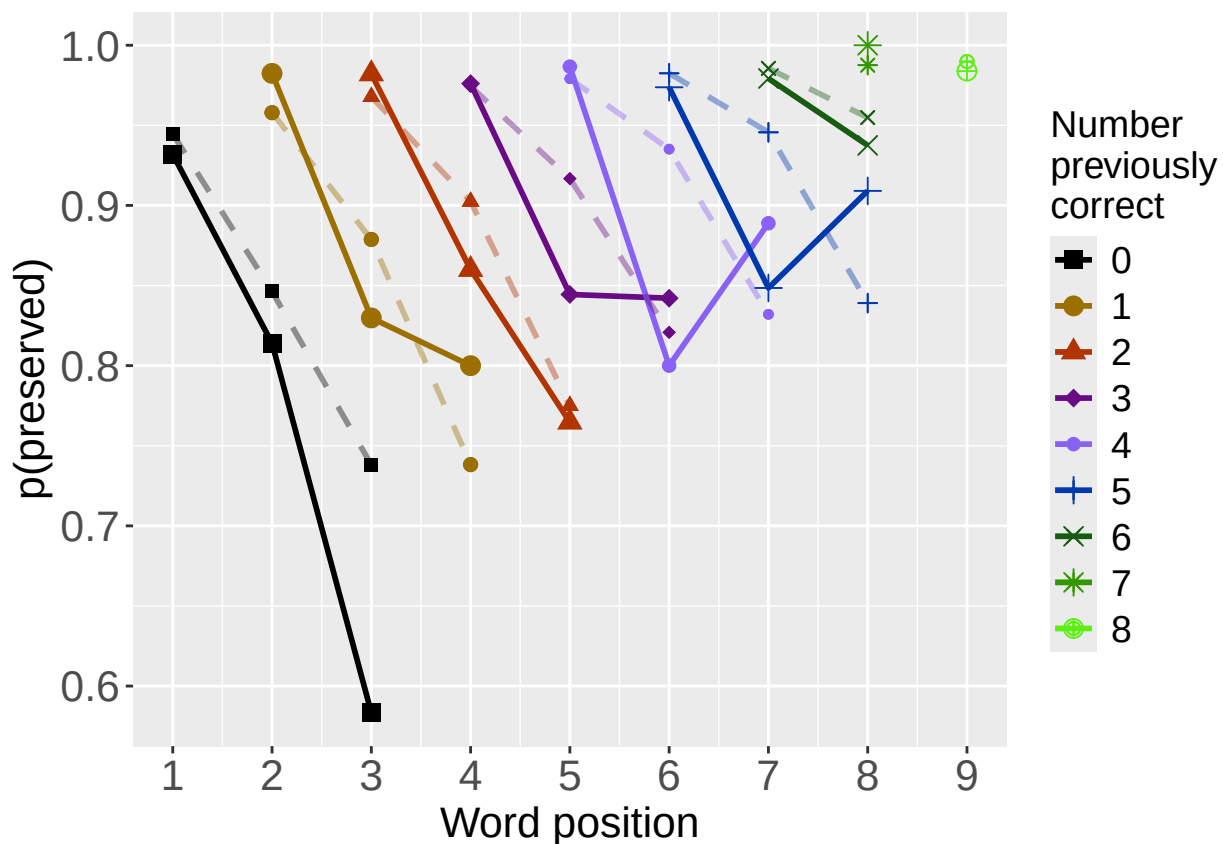
```
#####
# Single deletions from best model
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

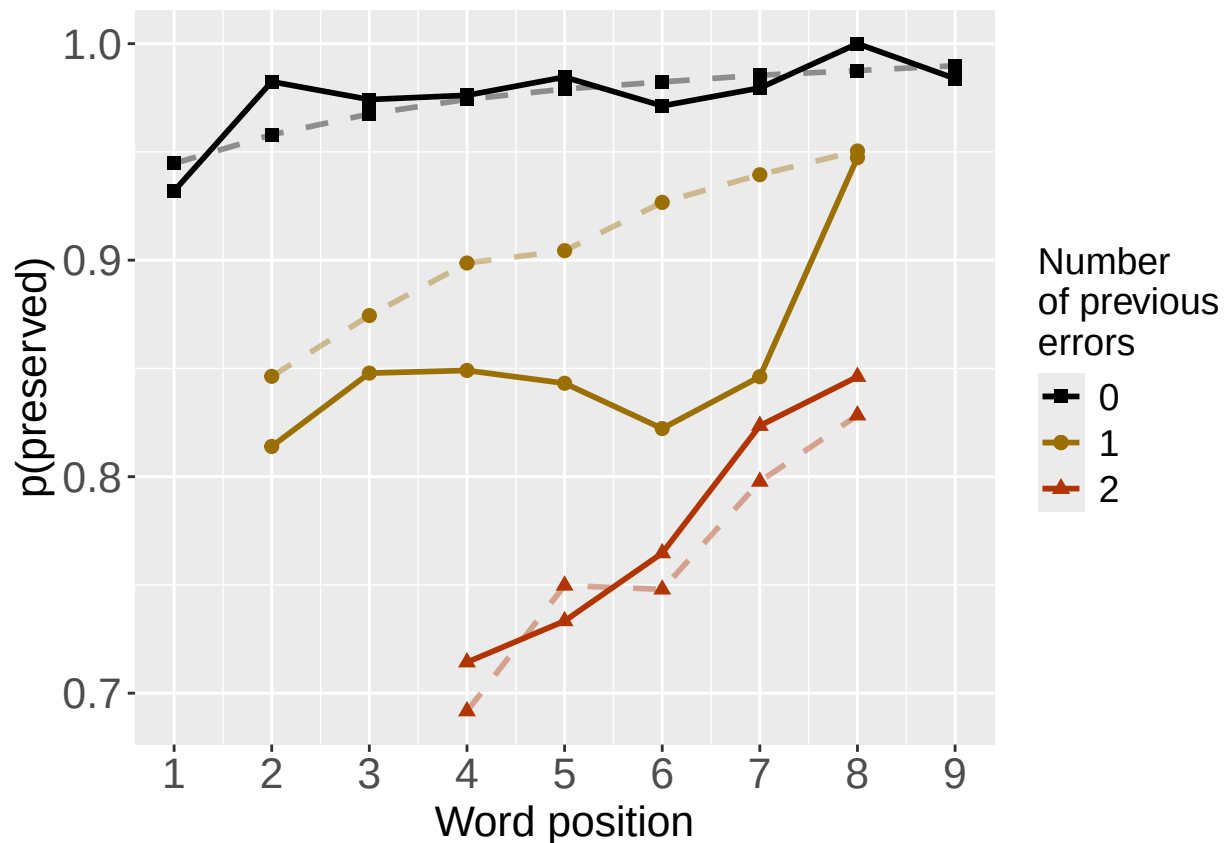
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          3.397      -1.138
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4116 Residual
## Null Deviance: 1534
## Residual Deviance: 1380 AIC: 1384
## log likelihood: -690.1023
## Nagelkerke R2: 0.1178387
## % pres/err predicted correctly: -330.4231
## % of predictable range [ (model-null)/(1-null) ]: 0.08104281

```



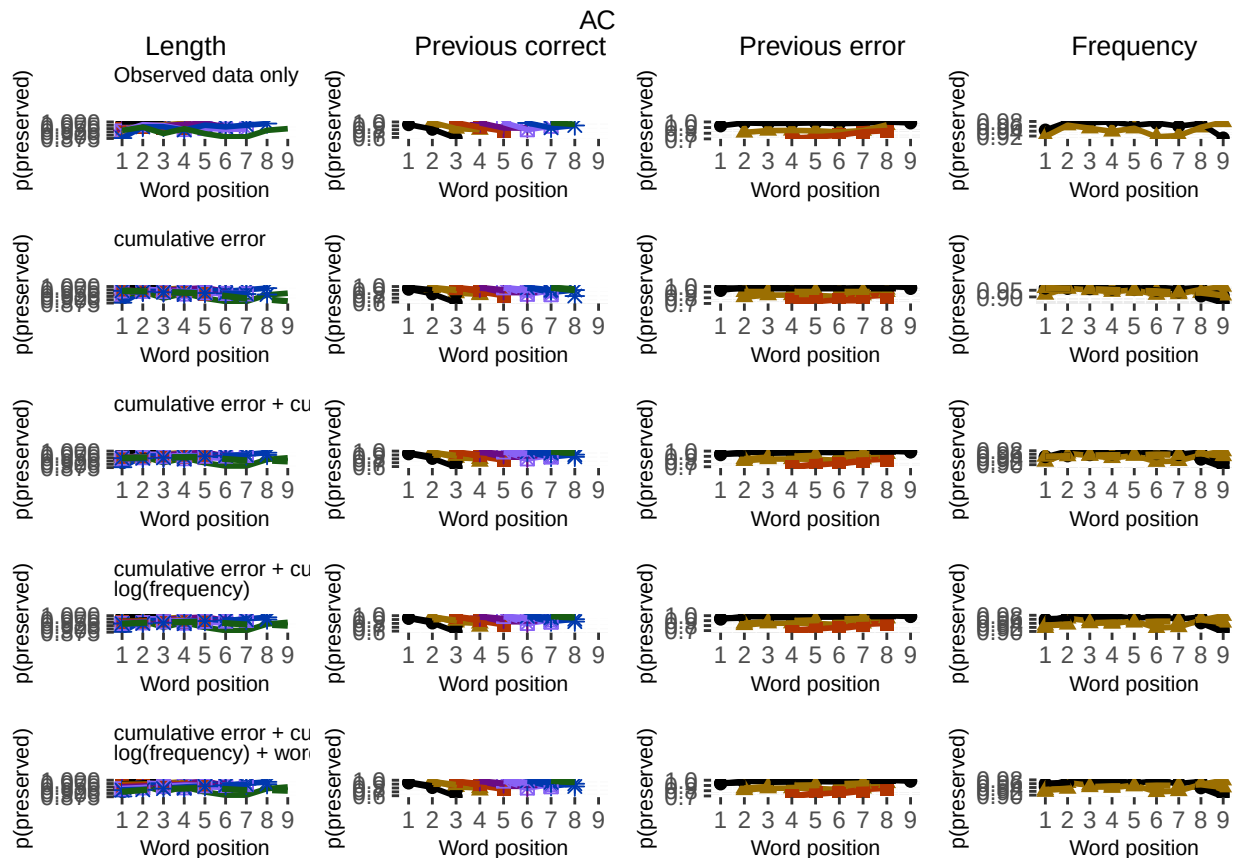
```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.8769      -1.1448      0.2324
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4115 Residual
## Null Deviance:      1534
## Residual Deviance: 1350 AIC: 1356
## log likelihood: -675.0204
## Nagelkerke R2: 0.140446
## % pres/err predicted correctly: -326.567
## % of predictable range [ (model-null)/(1-null) ]: 0.09173476
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      2.8963      -1.0976      0.2469      0.1909
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4114 Residual
## Null Deviance:      1534
## Residual Deviance: 1329 AIC: 1337
## log likelihood: -664.433
## Nagelkerke R2: 0.1562176
## % pres/err predicted correctly: -325.3066
## % of predictable range [ (model-null)/(1-null) ]: 0.09522951
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq      stimlen
##      3.9582      -1.0500      0.2777      0.1683      -0.1475
##
## Degrees of Freedom: 4117 Total (i.e. Null); 4113 Residual
## Null Deviance:      1534
## Residual Deviance: 1321 AIC: 1331
## log likelihood: -660.2549
## Nagelkerke R2: 0.1624192
## % pres/err predicted correctly: -324.2595
## % of predictable range [ (model-null)/(1-null) ]: 0.09813288
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

[illegible]

```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes c
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres	stimlen	log_freq
McFadden	0.0913907	0.0255432	0.0087027	0.0135074

	CumErr	CumPres	stimlen	log_freq
SquaredCorrelation	0.0331459	0.0092667	0.0031801	0.0049179
Nagelkerke	0.0331459	0.0092667	0.0031801	0.0049179
Estrella	0.0356966	0.0099824	0.0033571	0.0052459

	deviance	deviance_explained
CumErr + CumPres + log_freq + stimlen	1320.510	213.4400
CumErr + CumPres + log_freq	1328.866	205.0839
CumErr + CumPres	1350.041	183.9090
CumErr	1380.205	153.7453
null	1533.950	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained
## CumErr + CumPres + log_freq + stimlen CumErr + CumPres + log_freq + stimlen 1320.510      213.4400      13.91440
## CumErr + CumPres + log_freq              CumErr + CumPres + log_freq 1328.866      205.0839      13.36966
## CumErr + CumPres                        CumErr + CumPres 1350.041      183.9090      11.98925
## CumErr                                CumErr 1380.205      153.7453      10.02284
## null                                  null 1533.950       0.0000      0.00000
##                                percent_of_explained_deviance increment_in_explained
## CumErr + CumPres + log_freq + stimlen      100.00000      3.914964
## CumErr + CumPres + log_freq                96.08504      9.920746
## CumErr + CumPres                          86.16429     14.132190
## CumErr                                    72.03210     72.032101
## null                                      NA      0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + log_freq + stimlen	13.91440	100.00000	3.914964
CumErr + CumPres + log_freq	13.36966	96.08504	9.920746
CumErr + CumPres	11.98925	86.16429	14.132190
CumErr	10.02284	72.03210	72.032100
null	0.00000	NA	0.000000

```

                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.65621666
## CumPres   0.18346119
## stimlen   0.06295928
## log_freq  0.09736287

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

62

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

```

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)

sse_table

##               model p_accounted_for model_deviance diff_CumErr diff_CumErr+CumPres diff_CumErr+CumPres+log_fr
## 1      preserved ~ CumErr      0.8518102      1380.205  0.00000000      -0.069942474      -0.0756881
## 2      preserved ~ CumErr+CumPres      0.9217527      1350.041  0.06994247      0.000000000      -0.0057456
## 3      preserved ~ CumErr+CumPres+log_freq      0.9274983      1328.866  0.07568812      0.005745649      0.0000000
## 4 preserved ~ CumErr+CumPres+log_freq+stimlen      0.9313497      1320.510  0.07953949      0.009597015      0.0038513
## diff_CumErr+CumPres+log_freq+stimlen
## 1      -0.079539489
## 2      -0.009597015
## 3      -0.003851366
## 4      0.000000000

```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.8518102	1380.205
preserved ~ CumErr+CumPres	0.9217527	1350.041
preserved ~ CumErr+CumPres+log_freq	0.9274983	1328.866
preserved ~ CumErr+CumPres+log_freq+stimlen	0.9313497	1320.510

model	diff_CumErr	diff_CumErr+CumPres	diff_CumErr+CumPres+log_freq	diff_CumErr+CumPres+log_freq+stimlen
preserved ~ CumErr	0.0000000	-0.0699425	-0.0756881	-0.0795395
preserved ~ CumErr+CumPres	0.0699425	0.0000000	-0.0057456	-0.0095970
preserved ~ CumErr+CumPres+log_freq	0.0756881	0.0057456	0.0000000	-0.0038514
preserved ~ CumErr+CumPres+log_freq+stimlen	0.0795395	0.0095970	0.0038514	0.0000000

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
                                   CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```